

Your battery report says line 48 leaks a WakeLock.

EcoTrace shows you the alarm in `MainActivity` that put line 48 on the path ninety-six times a day — and what to change instead.

Every battery tool tells you **what** is draining you.

None of them tell you **why** — and "why" is the only part that is expensive to get wrong, because the fix is usually not where the symptom is.

```
MainActivity.onCreate()      the alarm nobody questioned
-> AlarmManager.setRepeating() wakes the device every 15 min, no backoff
-> SyncService.onStartCommand() 96 triggers a day
-> UserRepository.sync()        no cache hit, goes to network
-> NetworkManager.fetch()       no timeout set
-> wakeLock.acquire()           <-- the line the report points at
```

The `acquire()` is a symptom. The `setRepeating()` is the cause, and it is **three files and four hops upstream**. Patching the symptom moves the number and leaves the chain intact.

The gap is specifically **cross-file** reasoning.

TOOL	HOW IT READS YOUR CODEBASE	TRACES MULTI-FILE CHAINS
Android Profiler	Runtime only, no source analysis	No
SonarQube / Android Lint	Single-file pattern matching	No
LLM chat tools	Chunked or serially-read context; loses call edges past a few files	Partially
EcoTrace — local tracer	Builds a real call graph over every indexed source	Yes, offline
EcoTrace + IBM Bob 2.0	The same graph, plus full-repo reasoning over it	Yes, with a per-chain fix

The linters are not weak — they are answering a **narrower question**. Nobody is asking the wide one at the repository scale.

A desktop workbench that ranks, explains and grades your energy debt.

DETECT

23 detectors over every .java and .kt file, on comment-stripped source so every line number is the real one.

TRACE

A call graph over the whole project, walked backwards from each finding to the lifecycle method that put it there.

FIX

A remediation written for that chain — the modern scheduling API, not a patch of the symptom.

MEASURE

An ADB session attributes real drain to your package, at 0.1 mAh resolution.

GRADE

A weighted A–F with the weights shown, so the number is arguable rather than oracular.

REPORT

A self-contained document, typeset for A4, that prints to a real selectable PDF.

This is the actual output. Four files, one root cause.

1	ROOT CAUSE	onStartCommand() SyncService.java:42
2	PROPAGATED THROUGH	startSyncCycle() SyncEngine.java:31
3	PROPAGATED THROUGH	syncUserProfile() UserRepository.java:31
4	SYMPTOM	fetchUserProfile() NetworkManager.java:48 — acquire() found but release() is not in a finally block

WHAT A LINTER SAYS

"NetworkManager.java:48 — unclosed WakeLock."

Action: wrap in try/finally. The number improves. The chain does not.

WHAT ECOTRACE SAYS

The alarm in MainActivity.onCreate re-arms this 96× a day with no backoff and no Doze deferral.

Action: replace the service. The WakeLock leak becomes structurally impossible.

Try it now. Nothing to install.

The hosted build opens onto a populated workbench already — running the **real** 23 detectors, call graph, grader and report over EcoTrace's own rule-test corpus. No mock data, no API key, no sign-up.

23

DETECTORS RUN

4

HOPS IN THE DEEPEST CHAIN

17

SOURCES, 11 OF THEM CLEAN

The eleven clean files matter as much as the six flagged ones. They are what the coverage matrix renders as an explicit zero — "**checked and clean**" is a **result**, and a tool that only knows how to find things cannot show you it looked.

The artefact you attach to the ticket.

TEN SECTIONS

Verdict · ranked plan · score breakdown · at a glance · distribution · causal chain · all findings · **23-rule coverage matrix** · device profile · method and limits.

THREE WAYS OUT

Standalone HTML, PDF via the print pipeline, or open in a browser. 13 A4 pages, vector text, searchable.

IT IS HONEST ABOUT ITS OWN NUMBERS

- ✗ A drain rate from the charge level is printed **with its 30 mAh resolution** attached, because that is not a two-decimal measurement.
- ✗ A finding with no traced chain **says so** instead of omitting the chain.
- ✗ Method and limits names the blind spots: no reflection, no DI, no generated code.

This is aimed at an expert reader. The fastest way to lose one is a number they cannot trust.

Bob did not write the feature. Bob wrote the builder.

TIER 1 — ALWAYS ON, NO KEY

- ✓ 23 detectors + call-graph builder, **built in Bob sessions 04 and 06**
- ✓ Runs offline, reproducibly, on every scan
- ✓ Full product with Bob switched off

TIER 2 — IF YOU PASTE A KEY

- ✓ Revises severity and description per finding
- ✓ Writes a fix **for that specific chain**
- ✓ May only adjust a finding that already exists locally

The call-graph tracer and the 23-rule catalog are the output of the Bob sessions in `/bob_sessions`. Bob is the reason the tool exists; it is not a runtime dependency of the demo.

SHELL

Tauri 2.0 · Rust · 4 rooted commands, 2 MB file cap, no path escapes the project

ANALYSIS

TypeScript in a **Web Worker** — the window stays live during a scan

UI

IBM Carbon Design System, tokens generated from Carbon's theme maps

Run against production code, not a sample.

SOURCES 847 Signal Android	FINDINGS 12 1.4% of files	GRADE C 57 / 100	LONGEST CHAIN 7 files, traced
---	--	---	--

A low finding density is the point, not a disappointment: it means the tool is reporting on code that mostly gets it right, and the twelve it found are worth a team's afternoon.

WHAT THE BUILD HISTORY SHOWS

Five recorded scans on this repository moved the composite **D** → **C** and the finding count **21** → **12**, at a measured 2.1 → 1.4 mAh/min. That is the only honest measure of whether a refactor helped.

STATED LIMITATION

These figures come from a single manual run. No automated benchmark in this repository reproduces them, so they are an illustrative record rather than a measured claim.

A CI check, not a developer tool.

Nobody budgets for a lint tool. Teams budget for **store ratings, one-star reviews and battery complaints in support** — all of which are downstream of exactly these defects.

TODAY

Desktop app, \$29 per developer seat. Self-hostable. The report is the deliverable: it is what gets attached to the ticket and read by someone who will not install anything.

NEXT

A pull-request action that fails the build on a new critical finding and comments the traced chain on the diff. This is the wedge: it moves the cost from review time to before merge.

THEN

Fleet mode — the same rules over a portfolio of apps, ranked by energy debt, with the grade as the metric engineering leadership already reports on.

Why it scales: the 23 rules are a static corpus, so marginal cost per additional repository is a clone and a walk. The ADB layer is the part that does not generalise — and that is deliberate, because it is a diagnostic for the developer, not the product.

Knowing what drains the battery is not enough.

The expensive part of a battery bug is not finding it. It is finding it again in six weeks, because nobody changed the decision that caused it.

LIVE DEMO

Opens populated. No install, no key.

REPOSITORY

Public, with the Bob session record.

WINDOWS BUILD

Signed installer in Releases.